# 📘 Tutorial 5 — Realtime Listeners

> Continue the React + Firestore Todo series by making updates appear instantly.

---

## ✅ What You Will Build

A todo list powered by Firestore realtime listeners using `onSnapshot`.

When data changes in Firestore, your UI updates automatically.

---

## 🎯 Learning Goals

By completing this tutorial you will:

- Use Firestore realtime subscriptions
- Clean up subscriptions in `useEffect`
- Reduce manual refresh logic in React

---

## 🧰 Before You Start

You should already have:

- Tutorial 4 completed
- A working Todo component with `useEffect`

---

# Part 1 — Update Firestore Imports

Open:

```
src/components/Todo.js
```

Change imports from:

```
import { collection, addDoc, getDocs } from "firebase/firestore";
```

To:

```
import {
  collection,
  addDoc,
```

```
  onSnapshot,
  query,
  orderBy,
} from "firebase/firestore";
```

---

## Part 2 — Stop Calling `fetchTodos()` After Adds

In `addTodo`, remove the manual fetch call.

Use:

```
const addTodo = async (e) => {
  e.preventDefault();

  await addDoc(collection(db, "todos"), { todo });
  setTodo("");
};
```

✅ Realtime listener will handle updates automatically.

---

## Part 3 — Replace `getDocs` Logic with `onSnapshot`

If you have a `fetchTodos` function, you can remove it.

Update `useEffect` to:

```
useEffect(() => {
  const q = query(collection(db, "todos"), orderBy("todo", "asc"));

  const unsubscribe = onSnapshot(q, (snapshot) => {
    const list = snapshot.docs.map((doc) => ({
      id: doc.id,
      ...doc.data(),
    }));

    setTodos(list);
  });

  return () => unsubscribe();
}, []);
```

---

## Part 4 — Optional: Keep Input Controlled

---

To clear input reliably after adding, make the input controlled:

```
<input
  type="text"
  placeholder="What do you want to do?"
  value={todo}
  onChange={(e) => setTodo(e.target.value)}
/>
```

---

# Part 5 — Test Realtime Behavior

1. Open your app in two browser tabs
2. Add a todo in one tab
3. Watch the second tab update instantly

You should see:

✅ No manual refresh needed ✅ UI stays synced with Firestore changes

---

## 🛠️ Common Errors

### ❌ `onSnapshot is not defined`

Import `onSnapshot` from Firestore.

### ❌ Listener keeps running after navigation

Make sure your `useEffect` returns cleanup:

```
return () => unsubscribe();
```

---

## 📚 Docs

- Get realtime updates (Firestore)
- React `useEffect` reference

---

🎯 You have completed the core Todo app tutorial series.